

## SIMULACIÓN FORMACIÓN EN EMPRESAS

### PROYECTO

1º CFGS DAW. 2024/2025. IES TORREVIGÍA

Autor: Jesús Mañogil



**Reconocimiento – No Comercial – Compartir Igual (by-nc-sa)**

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

## Proyecto Formación en Empresa

|                                         |   |
|-----------------------------------------|---|
| 1. Presentación.....                    | 3 |
| 2. Requerimientos.....                  | 3 |
| 3. Análisis.....                        | 5 |
| 4. Diseño.....                          | 5 |
| 5. Implementación.....                  | 6 |
| 6. Pruebas.....                         | 7 |
| 7. Documentación.....                   | 7 |
| 8. Entrega.....                         | 8 |
| 9. Exposición.....                      | 8 |
| 10. Módulos de 1º DAW relacionados..... | 9 |

## 1. Presentación

---

En este proyecto haremos el desarrollo de una aplicación software que cubra las necesidades de una empresa en cuanto al almacenamiento y gestión de la información utilizada en su día a día.

El proyecto se realizará **individualmente** y cubrirá las fases de **análisis, diseño, implementación, pruebas y documentación**. Al final del mismo y como método de evaluación del trabajo realizado, se realizará una breve **exposición** de lo realizado centrada en presentar las principales decisiones de diseño y programación tomadas a lo largo del desarrollo.

## 2. Requerimientos

---

Una empresa dedicada al **alquiler de viviendas**, en concreto con nombre **Alquilaria**, acaba de empezar su actividad y nos ha pedido desarrollar una aplicación que permita almacenar y manejar toda la información que necesita.

Es importante mantener la información de los **propietarios**, en concreto su DNI, nombre, correo electrónico y teléfono. Aunque se pueden identificar con su DNI, nos comentan que prefieren además, poder identificarlos con un número.

La información sobre las **viviendas** que los propietarios ponen en alquiler también se debe almacenar. Se trata de su dirección, precio de alquiler mensual, superficie, una breve descripción y si el propietario acepta inquilinos con mascotas o no. La identificación de cada propiedad se hará mediante un código alfanumérico. También debemos anotar el tipo de propiedad entre las opciones: apartamento, ático y casa.

Con respecto a los **inquilinos**, además de los datos habituales como DNI, nombre, correo electrónico y teléfono, también necesitamos saber si tiene mascota. La identificación de los inquilinos se realizará mediante un número único que el sistema les asigne, como sucede con los propietarios.

El alquiler de la vivienda por el inquilino requiere de la firma de un **contrato**. Necesitaremos almacenar las fechas de inicio y fin, el precio, ya que podría ser diferente del que inicialmente conste en los datos de la propiedad y el estado que podría ser pendiente, activo o vencido. Inicialmente el contrato estará pendiente de firma y pago, pasando a estado activo. Cuando finalice el contrato pasará a estado vencido.

Como es lógico, un propietario puede tener una o más viviendas, aunque una vivienda tendrá solamente un propietario. Por otro lado, un inquilino podría alquilar diferentes viviendas en diferentes intervalos de tiempo.

En cuanto a las necesidades de gestión de la información referentes a los propietarios, inquilinos, viviendas y contratos, nos solicitan inicialmente las **funcionalidades**:

- Mantenimiento básico sobre los datos de **propietarios**, **viviendas** e **inquilinos**: Añadir, borrar, consultar por códigos y modificar.
- Mantenimiento de **contratos**: Borrado, modificación, consultas y cambio de estado. Cuando se venza la fecha del contrato automáticamente el sistema debe marcarlo como “Vencido”.
- **Estadísticas** / consultas avanzadas: Principalmente relacionadas con el histórico de alquileres realizados por los inquilinos y las viviendas que los propietarios están alquilando.
- Exportación de información: Es necesario que la aplicación pueda exportar a archivos formato JSON y CSV las consultas avanzadas del apartado anterior.

Es habitual en el proceso de desarrollo de software que la empresa no considere inicialmente determinadas funcionalidades o automatizaciones que el analista/desarrollador pueda tener en cuenta gracias a su experiencia. Por dicho motivo, el desarrollo estará abierto a incluir mejoras no incluidas en esta recogida de requerimientos. Un ejemplo de esto lo tenemos en las funcionalidades de “Estadísticas”. Esos cambios se defenderán en la exposición del proyecto.

### 3. Análisis

---

Partiendo de los requerimientos nos centraremos en determinar las principales funcionalidades de la aplicación y sus dependencias o conexiones con otras aplicaciones, si fuera el caso.

En este punto realizaremos el correspondiente **diagrama de secuencia** y el **diagrama de casos de uso**. Para este proyecto concreto, como se verá, esos diagramas son muy sencillos.

### 4. Diseño

---

En esta fase definimos cómo implementaremos el sistema para que cumpla con los requerimientos y el análisis realizado.

Es muy habitual dividir el diseño en dos partes: diseño de la **base de datos** y **arquitectura** de la aplicación. Además, se debe concretar en ese punto las **tecnologías**, **herramientas** y **patrones** de diseño que se emplearán.

#### A. Technology stack

Emplearemos lenguaje **Java** para realizar la implementación de la aplicación, en concreto su última versión de desarrollo **JDK 24**. Para conectarnos desde Java a la base de datos usaremos la librería **JDBC**.

Se valorará positivamente el uso de **Programación Orientada a Objetos** como enfoque de programación y el empleo del patrón de arquitectura **MVC** (Modelo-vista-controlador).

Las pruebas unitarias se desarrollarán empleando el framework de testing **JUnit**.

Se usará la utilidad **JavaDoc** para documentar nuestro código y se publicará en un servidor Web configurado en una máquina virtual o en un docker.

En cuanto al IDE que usaremos, se recomienda trabajar con **Visual Studio Code**.

La base de datos será **MySQL** y la instancia deberá estar instalada en un servidor independiente que esté ejecutándose en una máquina virtual o en un docker. Se recomienda usar el motor de almacenamiento InnoDB en lugar de MariaDB.

## B. Base de datos

La base de datos será del tipo **relacional** y su diseño seguirá los pasos habituales que deberán documentarse adecuadamente:

- **Diseño conceptual:** Se realizará un diagrama Entidad-Relación con las entidades, relaciones atributos y cardinalidades de las relaciones.
- **Diseño lógico:** En base al diagrama Entidad-Relación anterior se realizará un modelo lógico relacional indicando las tablas que contendrá la base de datos con sus campos y claves.
- **Diseño físico:** Crearemos la base de datos en nuestro servidor. Implementaremos un script de creación y otro de carga con unas primeras filas de datos que “pueblen” las tablas inicialmente.

En ese punto es cuando debemos tomar la decisión de establecer los límites de aquellas “reglas de negocio” o “funcionalidades” que llevaremos a nuestra base de datos y realizaremos mediante **programación SQL** o se codificarán desde el lenguaje que acceda a la base de datos, Java en nuestro caso. Por ejemplo, ¿insertaremos un cliente llamando a un procedimiento almacenado desde Java o emplearemos una instrucción SQL INSERT en Java para hacerlo ?

El diseño físico de la base de datos se puede considerar también como parte del propio desarrollo y no incluir dentro del apartado de diseño.

## C. Arquitectura

En caso del empleo de Programación Orientada a Objetos, definiremos el **diagrama de clases** de la aplicación empleando el language **UML**. Con ello concretamos las clases que usaremos cuando estemos en la fase de implementación.

## 5. Implementación

---

Como se ha indicado, la aplicación se desarrollará en Java, accediendo a una base de datos MySQL mediante JDBC. El acceso a esa base de datos se realizará mediante una conexión empleando la librería JDBC y un usuario de base de datos, diferente a root, con los mínimos permisos (privilegios) que la aplicación necesite.

Las recomendaciones más importantes serían:

- Los archivos de código del proyecto se distribuirán en tres carpetas siguiendo el patrón MVC: Controller, Model y View.
- En un archivo de configuración se guardarán aquellos parámetros configurables en la aplicación que consideremos, por ejemplo, los datos para conectarnos al servidor de base de datos mediante JDBC.
- Para las exportaciones de datos a ficheros formato JSON y CSV, se recomienda guardarlos en la carpeta de descargas de la máquina donde se ejecute la aplicación.
- Muy aconsejable el uso efectivo de comentarios en nuestro código. Es más importante explicar el porqué lo hacemos de ese modo, en lugar de ratificar lo que hace.
- Antes de la entrega del código se debe haber realizado una tarea previa de **refactorización** que mejore la comprensión y el futuro mantenimiento del trabajo realizado.

## 6. Pruebas

---

En primer lugar, empleando JUnit, crearemos **test unitarios** que prueben los métodos de las principales **clases** de nuestra aplicación. Todo el código generado para estas pruebas también se debe entregar.

Una vez creadas las pruebas unitarias de nuestras clases, escogeremos a un compañero para que realice las llamadas **pruebas de sistema**, confirmando que la aplicación funciona como se espera.

## 7. Documentación

---

Empleando JavaDoc documentaremos todas las clases de nuestro proyecto. Esa documentación generada, además de incluirla en la entrega, se subirá a un servidor Web.

Se realizará también un archivo formato Markdown con nombre README.md y contenido en **inglés**, que acompañará al código entregado y explicará brevemente el funcionamiento de la aplicación.

## 8. Entrega

---

Todo el trabajo realizado, incluido el código, documentación, pruebas, etc. se subirá a un repositorio en una cuenta GitHub que se compartirá únicamente con los responsables de su evaluación.

Una vez finalizada la fecha de entrega del proyecto, no se podrá realizar cambios en el repositorio GitHub hasta que se evalúe.

## 9. Exposición

---

El desarrollo del proyecto se defenderá ante los responsables de su evaluación, usando para ello una presentación en inglés con páginas web alojadas en el mismo servidor Web donde se ubicó la documentación generada por JavaDoc.

La exposición deberá seguir el siguiente guión:

1. Presentación
2. Decisiones de diseño destacables
3. Breve ejemplo de funcionamiento
4. Partes del desarrollo que han resultado más interesantes

## 10. Módulos de 1º DAW relacionados

En la siguiente tabla se puede comprobar la relación de módulos de 1º del ciclo DAW cuyos contenidos se tratan en este proyecto.

| Módulo                                                   | Apartado                                                                                                                                         |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Programación                                             | 4. Diseño: MVC<br>5. Implementación: Java y POO<br>6. Pruebas: JUnit                                                                             |
| Bases de datos                                           | 4. Diseño: Creación base de datos, programación SQL<br>5. Implementación: Consultas SQL                                                          |
| Entornos de desarrollo                                   | 3. Análisis: Diagrama de secuencia y Diagrama casos de uso<br>4. Diseño: IDE, Arquitectura                                                       |
| Lenguajes de marcas y sistemas de gestión de información | 5. Implementación: Exportación JSON y CSV<br>7. Documentación: Formato Markdown, documentación JavaDoc<br>9. Exposición: HTML, CSS, servidor Web |
| Sistemas informáticos                                    | 4. Diseño: Máquinas virtuales, Docker, Servidores                                                                                                |
| Inglés profesional                                       | 7. Documentación: Archivo descripción aplicación en inglés<br>9. Exposición: Presentación en inglés                                              |
| Itinerario personal para la empleabilidad I              | 1. Presentación: Perfil profesional full-stack developer salidas profesionales                                                                   |